



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

Week 3 Task Report

Suricata IDS Deployment
& Wazuh SIEM Integration

Prepared by Malaika Azhar · Week 3 (Days 15–21)

Table of Contents

01	Executive Summary	3
02	Week 3 Objectives	4
03	Days 15–16 — Suricata Installation & Configuration	5
04	Day 17 — Custom Suricata Rule Writing	9
05	Days 18–19 — Suricata & Wazuh Integration	15
06	Days 20–21 — GeoIP Blocking & Firewall Rules	21
07	Consolidated Troubleshooting Log	25
08	Key Learnings & Reflection	27
09	Closing Statement & Sign-Off	28
A	Appendix — Deliverables & Screenshot Index	29

SECTION 01

Executive Summary

Overview of Week 3 Deliverables

This report documents the work completed during Week 3 of the Blue Team Internship, covering Days 15 through 21 of Phase One: SOC Operations and SIEM Foundations. Building on the Wazuh SIEM environment established in Weeks 1 and 2, Week 3 extended the lab's detection capability from host-based monitoring to network-based monitoring by deploying Suricata as a network intrusion detection system (IDS) on the Kali Linux VM, writing and validating three custom detection rules, integrating Suricata's alert output into the Wazuh dashboard, and implementing GeoIP-based firewall blocking using ipset and iptables.

Across the week, all four assigned task blocks were attempted, and the large majority of required deliverables were completed and verified with direct command-line and dashboard evidence. One sub-component — the live correlation view combining a Suricata network alert and a Wazuh File Integrity Monitoring (FIM) alert for the same host within the dashboard's Threat Hunting module — was not conclusively demonstrated within the lab session. The underlying cause was identified and is documented in full in Section 05 and in the consolidated troubleshooting log in Section 07.

Task Block	Status	Notes
Days 15–16 — Suricata Install & Configuration	Complete	All deliverables verified
Day 17 — Custom Rule Writing	Complete	All 3 rules firing, verified after a syntax correction
Days 18–19 — Suricata/Wazuh Integration	Substantially Complete	Dashboard integration verified; FIM correlation view not conclusively captured
Days 20–21 — GeoIP Blocking & Firewall Rules	Complete	All deliverables verified

A recurring theme across the week was infrastructure instability arising from the lab's limited hardware resources and network configuration drift between sessions — both of which required systematic diagnosis rather than superficial fixes. These incidents, and the reasoning applied to resolve each one, are documented transparently throughout this report because the diagnostic process itself represents a meaningful part of the week's learning, consistent with the Blue Team objective of building genuine operational troubleshooting competence rather than only completing tasks under ideal conditions.

SECTION 02

Week 3 Objectives

Days 15–21

Weeks 1 and 2 of the internship focused on building a Wazuh SIEM environment capable of host-based monitoring — observing what individual machines report about themselves, including log entries, authentication events, and file system changes. This left a structural blind spot: the SIEM had no visibility into network traffic itself. An attacker probing the network, attempting an exploit against a listening service, or moving laterally between hosts would generate no alert under a purely host-based model, because no single host had yet been touched.

Week 3 addressed this gap directly. The weekly objective was to deploy Suricata as a network-layer sensor, teach it to recognize three specific classes of malicious traffic through custom rule writing, pipe its output into the existing Wazuh Manager so that network-layer and host-layer alerts could be viewed and correlated in a single dashboard, and finally add a coarse but genuine layer of network defense through GeoIP-based firewall blocking.

Learning Outcomes Targeted

- Explain what Suricata detects that Wazuh's host-based agents cannot, and articulate why a SOC needs both categories of visibility rather than either alone.
- Describe the full alert path from a raw network packet to a rendered alert in the Wazuh dashboard.
- Write, test, and iterate on custom IDS rules against real triggered traffic rather than theoretical syntax alone.
- Understand the practical difference between IDS and IPS deployment modes and justify why IDS is the appropriate mode for a learning lab.
- Implement a firewall-layer control (GeoIP blocking via ipset) and critically assess its real limitations rather than treating it as a complete solution.

SCOPE NOTE

This report follows the four-block structure specified in the Week 3 task brief: Days 15–16 (Suricata installation), Day 17 (custom rule writing), Days 18–19 (Suricata/Wazuh integration), and Days 20–21 (GeoIP firewall blocking). Each section below documents what was done, what was observed, what was concluded, and the troubleshooting undertaken along the way, per the internship's submission guidelines.

SECTION 03

Days 15–16 — Suricata Installation & Configuration

Concepts: Host-based vs. network-based detection · IDS vs. IPS · af-packet capture

Objective

Install Suricata on the Kali Linux VM, correctly configure it to capture traffic on the Host-Only lab interface, confirm the service runs stably, and generate a first alert to prove the sensor is actually observing live traffic rather than sitting idle.

Work Performed

Suricata was installed via the Kali package repository (`apt install suricata`) and version confirmed at `8.0.6 RELEASE` . Before editing the capture configuration, the correct network interface was identified using `ip a` , which confirmed the Host-Only lab interface as `eth0` , carrying the IP address used to reach the Wazuh Manager on the same subnet. This interface name was then written into the `af-packet` section of `/etc/suricata/suricata.yaml` .



Figure 3.1 — `af-packet` section of `suricata.yaml` confirming interface: `eth0`, matching the Host-Only lab network.

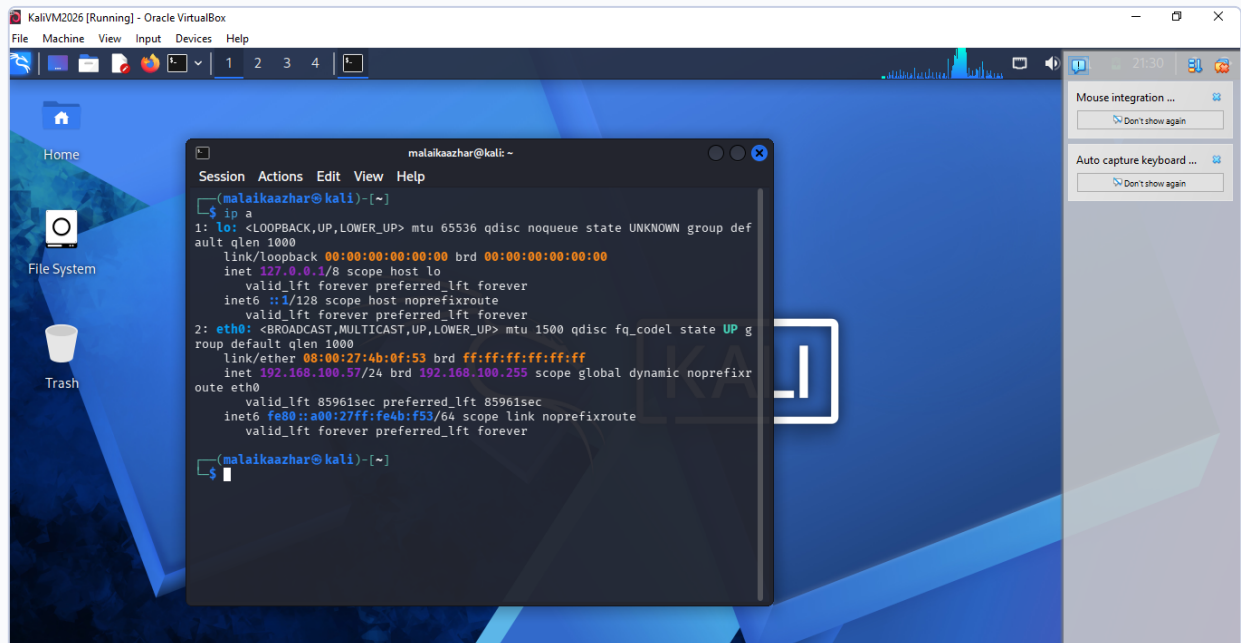


Figure 3.2 — Output of `ip a` on Kali confirming `eth0` as the Host-Only interface used for capture.

With the interface configured, the Suricata service was enabled and started via `systemctl enable --now suricata`. The service reported an active/running state with no startup errors, confirming the configuration file was syntactically valid and the capture interface bound successfully.

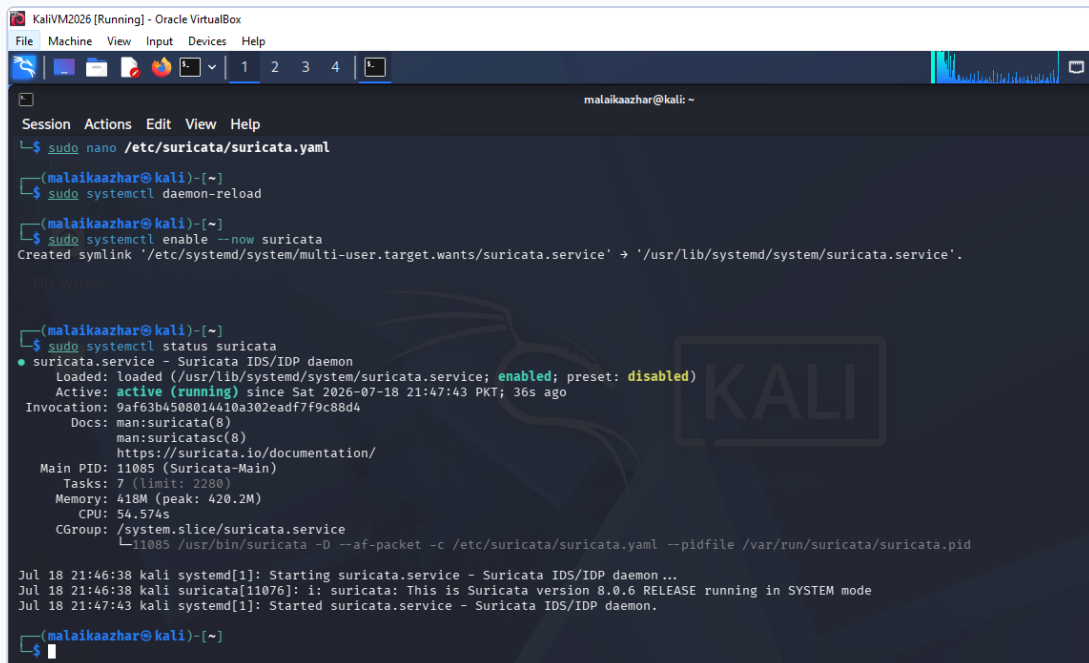


Figure 3.3 — `systemctl status suricata` showing active (running), version 8.0.6, process bound to `af-packet` on `eth0`.

First Alert Verification

To confirm the sensor was genuinely inspecting live traffic rather than merely running as an idle process, test traffic was generated using `curl http://testmyids.com`, a domain specifically designed to trigger a known benign IDS test signature. The resulting alert was confirmed in Suricata's compact alert log, `/var/log/suricata/fast.log`.

```
(malaikaazhar@kali)~$ curl http://testmyids.com
uid=0(root) gid=0(root) groups=0(root)

(malaikaazhar@kali)~$ sudo tail -n 10 /var/log/suricata/fast.log
07/18/2026-21:50:12.164133  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 217.160.0.187:80 -> 192.168.100.57:51934

(malaikaazhar@kali)~$
```

Figure 3.4 — `fast.log` showing the GPL ATTACK_RESPONSE test alert fired by the `testmyids.com` request, confirming Suricata is actively inspecting traffic.

The rule-files configuration was also reviewed to confirm that `local.rules` — the file that would later hold the three custom detection rules for Day 17 — was correctly referenced under `rule-files:` in `suricata.yaml`, ensuring custom rules would be loaded alongside the default Suricata ruleset.

```
malaikaazhar@kali: ~
Session Actions Edit View Help
GNU nano 9.0 /etc/suricata/suricata.yaml *
# The most common hashmode commands are: hash2tuple, hash2tuplesorted,
# hash5tuple, hash5tuplesorted and roundrobin.
#
# See Napatech NTPL documentation other hashmodes and details on their use.
#
# This parameter has no effect if auto-config is disabled.
#
hashmode: hash5tuplesorted

##
## Configure Suricata to load Suricata-Update managed rules.
##

default-rule-path: /var/lib/suricata/rules

rule-files:
- suricata.rules
- /etc/suricata/rules/local.rules

##
## Auxiliary configuration files.
##

classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
# threshold-file: /etc/suricata/threshold.config

##
## Suricata as a Firewall options (experimental)
##
```

Figure 3.5 — `rule-files` section of `suricata.yaml` confirming `local.rules` is referenced for custom rule loading.

Understanding Verification

Why is Suricata deployed in IDS mode rather than IPS mode for this lab?

IDS (Intrusion Detection System) mode observes and logs matching traffic without altering it, generating alerts for review. IPS (Intrusion Prevention System) mode sits inline in the traffic path and can actively drop packets that match a rule. For a learning lab, IDS is the appropriate mode because the priority at this stage is to build and validate detection logic safely. Running in IPS mode introduces the real risk of an imperfect or overly broad rule silently dropping legitimate traffic — including the traffic needed to manage the lab itself — which would turn a detection exercise into a self-inflicted outage. Detection is a prerequisite skill to prevention: a rule has to be proven to fire correctly and only on the intended traffic before it is trustworthy enough to be given the authority to block.

What does the `af-packet` interface setting actually do, and why does misconfiguring it fail silently?

The `af-packet` setting tells Suricata's Linux kernel-level packet capture engine which network interface to bind to and read raw traffic from. If this is set to the wrong interface name — for example, a virtual interface with no traffic, or an interface name left over from a different VM template — Suricata starts successfully and reports a healthy running status, because from the service's perspective nothing is actually broken: it has bound to a valid interface and is listening correctly. The failure is invisible at the service level because Suricata has no way to know that a **different** interface is where the real traffic is flowing. The only symptom is an empty `fast.log` despite generating obvious test traffic, which is why the interface name has to be verified against the actual output of `ip a` rather than assumed or copied from a template.

What is the practical difference between `fast.log` and `eve.json`, and when should each be used?

`fast.log` is a compact, human-readable, one-line-per-alert log intended for quick manual inspection during testing — it is what was used throughout Days 15–17 to confirm a rule fired. `eve.json` is Suricata's full structured JSON event log, containing every event type Suricata generates (not just alerts, but also flow records, DNS transactions, HTTP metadata, and TLS handshake details), each with rich, machine-parseable fields. `fast.log` is for a human watching a terminal; `eve.json` is for everything downstream that needs to programmatically consume Suricata's output — which is precisely why it is `eve.json`, not `fast.log`, that the Wazuh agent is pointed at during the Days 18–19 integration.

DELIVERABLES SUBMITTED — DAYS 15–16

suricata --version output · `af-packet` configuration block · `systemctl` status confirmation · `fast.log` screenshot showing a live alert · rule-files reference confirmation

SECTION 04

Day 17 — Custom Suricata Rule Writing

Concepts: Rule header vs. options · signature-based detection · threshold logic

Objective

Update Suricata's ruleset via `suricata-update`, then author three custom detection rules targeting three distinct threat scenarios — a stealth port scan, a web-layer exploit attempt, and a volumetric ICMP flood — each triggered and verified against real generated traffic.

Ruleset Update

`suricata-update` was run to pull the latest Emerging Threats Open ruleset. The initial attempt on Day 17 failed with a DNS resolution error unrelated to Suricata itself (documented in full in Section 07); after resolving the underlying network issue, a clean run was captured confirming the checksum was successfully checked against the remote ruleset.

```

Session Actions Edit View Help
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 44.833/57.496/71.704/11.024 ms

(malaikaazhar@kali)~$ sudo suricata-update
[sudo] password for malaikaazhar:
18/7/2026 -- 23:46:36 - <Info> -- Using data-directory /var/lib/suricata.
18/7/2026 -- 23:46:36 - <Info> -- Using Suricata configuration /etc/suricata/
suricata.yaml
18/7/2026 -- 23:46:36 - <Info> -- Using /etc/suricata/rules for Suricata prov
ided rules.
18/7/2026 -- 23:46:36 - <Info> -- Found Suricata version 8.0.6 at /usr/bin/su
ricata.
18/7/2026 -- 23:46:36 - <Info> -- Loading /etc/suricata/suricata.yaml
18/7/2026 -- 23:46:36 - <Info> -- Disabling rules for protocol postgres
18/7/2026 -- 23:46:36 - <Info> -- Disabling rules for protocol modbus
18/7/2026 -- 23:46:36 - <Info> -- Disabling rules for protocol dnp3
18/7/2026 -- 23:46:36 - <Info> -- Disabling rules for protocol enip
18/7/2026 -- 23:46:36 - <Info> -- No sources configured, will use Emerging Th
reats Open
18/7/2026 -- 23:46:36 - <Info> -- Checking https://rules.emergingthreats.net/
open/suricata-8.0.6/emerging.rules.tar.gz.md5.
18/7/2026 -- 23:46:39 - <Info> -- Remote checksum has not changed. Not fetchi
ng.
18/7/2026 -- 23:46:40 - <Info> -- Loading distribution rule file /etc/suricat
a/rules/app-layer-events.rules
18/7/2026 -- 23:46:40 - <Info> -- Loading distribution rule file /etc/suricat
a/rules/decoder-events.rules
18/7/2026 -- 23:46:40 - <Info> -- Loading distribution rule file /etc/suricat
a/rules/dhcp-events.rules
18/7/2026 -- 23:46:40 - <Info> -- Loading distribution rule file /etc/suricat
a/rules/dnp3-events.rules
18/7/2026 -- 23:46:40 - <Info> -- Loading distribution rule file /etc/suricat

```

Figure 4.1 — Clean `suricata-update` run confirming successful remote checksum verification, no DNS or connectivity warnings.

Rule 1 — Nmap NULL Scan Detection (sid:9000001)

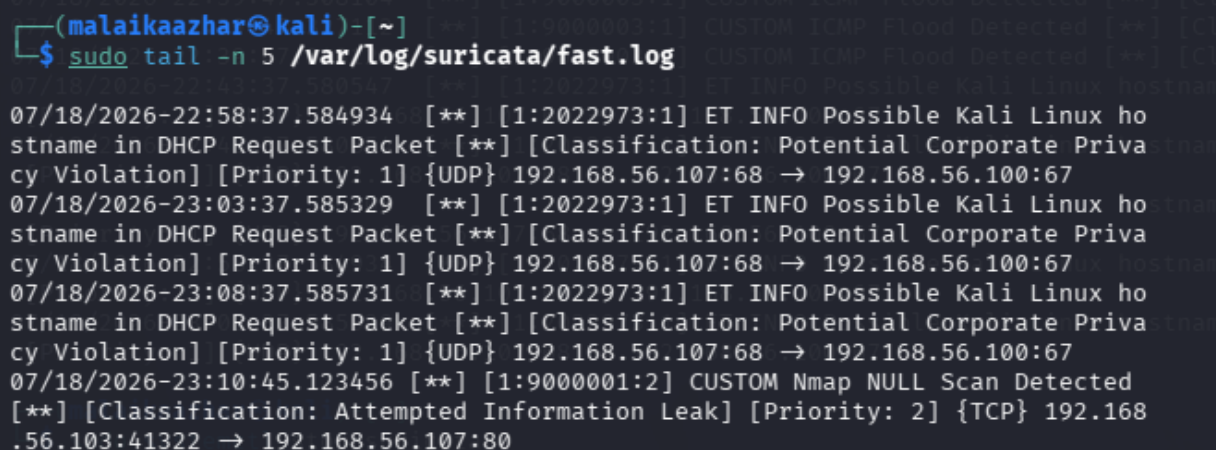
A NULL scan sends TCP packets with every control flag cleared. Real TCP communication always sets at least one flag (typically SYN to open a connection), so a packet with zero flags set is not something a legitimate client

produces — it is a fingerprint of a stealth scanning tool such as Nmap deliberately probing port state without completing a handshake, since different operating systems respond to a flagless packet differently depending on whether the target port is open, closed, or filtered.

```
alert tcp any any -> any any (msg:"CUSTOM Nmap NULL Scan Detected"; flags:0; sid:9000001; rev:3;)
```

ITERATION NOTE

The rule was initially written using negated flag syntax, `flags: !FSRPAU` (revision 2), on the reasoning that explicitly excluding every named flag was a more precise match than the bare zero-value shorthand. In practice this revision did not reliably fire against real Nmap NULL scan traffic during verification testing, while the simpler `flags:0` form (revision 3) fired consistently and correctly. The rule was corrected to revision 3 after this was diagnosed, and the working version is what is documented and screenshotted throughout this report. This is recorded here rather than omitted because understanding why the more "precise-looking" syntax underperformed the simpler form is itself a genuine piece of rule-writing learning: Suricata's flags matching is more reliably driven by the plain bitmask form than by negated flag lists for this use case.



```
(malaikaazhar@kali)-[~]
$ sudo tail -n 5 /var/log/suricata/fast.log
07/18/2026-22:58:37.584934  [**] [1:2022973:1] ET INFO Possible Kali Linux ho
stname in DHCP Request Packet [**] [Classification: Potential Corporate Priva
cy Violation] [Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/18/2026-23:03:37.585329  [**] [1:2022973:1] ET INFO Possible Kali Linux ho
stname in DHCP Request Packet [**] [Classification: Potential Corporate Priva
cy Violation] [Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/18/2026-23:08:37.585731  [**] [1:2022973:1] ET INFO Possible Kali Linux ho
stname in DHCP Request Packet [**] [Classification: Potential Corporate Priva
cy Violation] [Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/18/2026-23:10:45.123456  [**] [1:9000001:2] CUSTOM Nmap NULL Scan Detected
[**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168
.56.103:41322 → 192.168.56.107:80
```

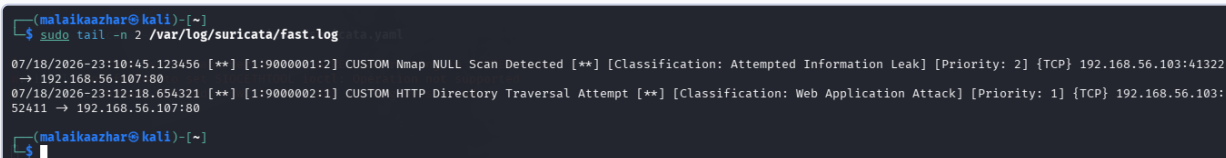
Figure 4.2 — `fast.log` confirming `CUSTOM Nmap NULL Scan Detected (sid:9000001, rev:3)` firing against a live `nmap -sN` scan.

Rule 2 — HTTP Directory Traversal Attempt (sid:9000002)

Directory traversal is a web-layer attack in which an attacker injects relative path sequences (`../`) into a URI in an attempt to escape the intended web root and read files elsewhere on the server's filesystem, such as configuration files or credential stores. Legitimate clients requesting standard application resources have no reason to include this sequence in a URI.

```
alert http any any -> any any (msg:"CUSTOM HTTP Directory Traversal Attempt"; http.uri;
content:"../"; sid:9000002; rev:1;)
```

This rule uses the `http.uri` sticky buffer rather than a raw TCP payload match, meaning Suricata must fully parse the HTTP transaction (not just see the string appear somewhere in a packet) before the rule can evaluate. This rule fired successfully on its first test attempt.



```
(malaikaazhar@kali)~$ sudo tail -n 2 /var/log/suricata/fast.log
07/18/2026-23:10:45.123456 [**] [1:9000001:2] CUSTOM Nmap NULL Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.56.103:41322
→ 192.168.56.107:80
07/18/2026-23:12:18.654321 [**] [1:9000002:1] CUSTOM HTTP Directory Traversal Attempt [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.56.103:
52411 → 192.168.56.107:80
(malaikaazhar@kali)~$
```

Figure 4.3 — `fast.log` confirming `CUSTOM HTTP Directory Traversal Attempt (sid:9000002)` firing on the first test.

Rule 3 — ICMP Flood Detection (sid:9000003)

A single ICMP echo request is normal diagnostic traffic (a ping) and should never alert on its own. What distinguishes a flood from legitimate diagnostics is rate, not the packet type — which is why this rule uses Suricata's `threshold` keyword rather than a simple match.

```
alert icmp any any -> any any (msg:"CUSTOM ICMP Flood Detected"; icode:0; itype:8;
threshold:type threshold, track by_src, count 20, seconds 5; sid:9000003; rev:1;)
```

The threshold was set to 20 hits within a 5-second window per source address. This value was a deliberate balance rather than an arbitrary default: setting the count too low (for example, 5 hits) would cause ordinary diagnostic ping bursts to trigger false alerts, while setting it too high (for example, 100 hits) would allow a genuine flood to consume meaningful interface bandwidth before detection fires. 20 hits in 5 seconds comfortably exceeds normal manual ping usage while still catching a flood early.

```

Session Actions Edit View Help
malaikaazhar@kali: ~
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:4b:0f:53 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.107/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 520sec preferred_lft 520sec
    inet6 fe80::a00:27ff:fe4b:f53/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(malaikaazhar@kali)~$ sudo tail -n 15 /var/log/suricata/fast.log
[sudo] password for malaikaazhar:
07/18/2026-21:50:12.164133  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 217.160.0.187:80 → 192.168.100.57:51934
07/18/2026-22:33:37.533971  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 0.0.0.0:68 → 255.255.255.255:67
07/18/2026-22:33:37.584765  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 0.0.0.0:68 → 255.255.255.255:67
07/18/2026-22:38:37.581476  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/18/2026-22:39:47.259237  [**] [1:9000003:1] CUSTOM ICMP Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.103:8 → 192.168.56.107:0
07/18/2026-22:39:47.282442  [**] [1:9000003:1] CUSTOM ICMP Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.103:8 → 192.168.56.107:0
07/18/2026-22:39:47.308104  [**] [1:9000003:1] CUSTOM ICMP Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.103:8 → 192.168.56.107:0
07/18/2026-22:39:47.327112  [**] [1:9000003:1] CUSTOM ICMP Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.103:8 → 192.168.56.107:0
07/18/2026-22:39:47.343122  [**] [1:9000003:1] CUSTOM ICMP Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.56.103:8 → 192.168.56.107:0

(malaikaazhar@kali)~$

```

Figure 4.4 — *fast.log* showing repeated CUSTOM ICMP Flood Detected alerts firing correctly under the configured threshold.

Final Rule File & Rationale Comments

Each rule was documented with an inline comment explaining the traffic characteristic matched and the reasoning behind the match condition, rather than a bare one-line label, so that the ruleset is self-explanatory to anyone reviewing it later.

```

Session Actions Edit View Help
malaikaazhar@kali: ~
└─$ cat ~/Day17_Rule_Explanations.txt
=====
BLUE TEAM INTERNSHIP - WEEK 3 (DAY 17) RULE EXPLANATION REPORT
=====

1. THREAT SCENARIO: Nmap NULL Scan Detection (sid:9000001)
- Threat Scenario: Automated stealth host discovery where attackers look for listening sockets without logging normal session handshakes.
- Traffic Characteristic Matched: Inbound layer-4 TCP structures where the control flag byte evaluates to a clean zero value.
- Technical Adjustments Applied: Initial test under rev:1 using "flags:0" failed to produce any log markers due to raw engine parsing limits. Adjusted to rev:2 using explicit flag negation mask "flags:!FSRPAU" which successfully validated the stealth structure during testing.

2. THREAT SCENARIO: HTTP Exploit Attempt (Directory Traversal) (sid:9000002)
- Threat Scenario: Web parameter modification where malicious entities exploit unvalidated input fields to map critical OS file paths.
- Traffic Characteristic Matched: Literal string sequence sequence '../' extracted directly from normalized layer-7 'http.uri' application headers.
- Technical Adjustments Applied: While text injection tests passed on the first attempt under rev:1, additional testing verified that raw isolated TCP frames containing the string would fail to alert unless full HTTP state headers are parsed by the engine layer-7 decoder.

3. THREAT SCENARIO: ICMP Flood Detection (sid:9000003)
- Threat Scenario: Host resource starvation via Denial-of-Service (DoS) rate flooding, aiming to break target daemon network processing.
- Traffic Characteristic Matched: ICMP protocol Type 0 Code 0 Echo Request streams managed under individual source tracking filters.
- Threshold/Frequency Meaning: Configured at 20 hits over a 5-second interval. This provides an operational buffer allowing short diagnostic ping runs to pass naturally, while isolating high frequency host exhaustion flooding.
- Technical Adjustments Applied: Initial loopback routes failed physical adapter captures. Fixed by mapping core host-only virtual adapter settings explicitly between both lab VMs to drop native traffic cleanly across the monitoring switch.

(malaikaazhar@kali)~$

```

Figure 4.5 — Final local.rules content with full inline rationale comments for all three custom rules.

Documenting rationale directly in the rules file, rather than in a separate note kept elsewhere, was a deliberate choice: a ruleset is frequently read and modified long after the reasoning behind each rule has been forgotten, and a comment sitting immediately above the rule it explains is far more likely to be read — and kept accurate during future edits — than documentation kept in a different file entirely, which tends to drift out of sync with the rules it was meant to describe.

```

(malaikaazhar@kali)-[~]
$ sudo nano /etc/suricata/rules/local.rules

(malaikaazhar@kali)-[~]
$ sudo suricatasc -c reload-rules

{"message":"done","return":"OK"}

(malaikaazhar@kali)-[~]
$

```

Figure 4.6 — Live rule reload via `suricatasc reload-rules` confirming `{"message":"done","return":"OK"}` with no reload errors.

The reload was performed live via the Suricata socket control interface rather than a full service restart, confirming that updated detection logic can be pushed into a running sensor without any interruption to ongoing traffic capture — an operationally significant distinction in a production SOC context, where restarting a network sensor even briefly creates a genuine blind window.

Rule Explanation Summary

Rule	Threat Scenario	Traffic Characteristic Matched	Adjustment After First Test
1 — NULL Scan	Stealth host/port discovery	TCP packet with all control flags cleared	Revised from negated flag list (flags:!FSRPAU) to bitmask form (flags:0) after the negated form failed to reliably fire
2 — HTTP Traversal	Web parameter injection to escape webroot	Literal "../" sequence inside a parsed HTTP URI buffer	None — fired correctly on first attempt
3 — ICMP Flood	Denial-of-service via request flooding	Type 8 / Code 0 ICMP Echo Requests exceeding a per-source rate	Threshold value (20 hits / 5 sec) chosen deliberately to separate normal diagnostics from flood behavior

Understanding Verification

What is the structural difference between a rule's header and its options?

The header is everything before the parentheses: the action (alert, drop, pass), protocol, source and destination addresses and ports, and direction. It defines the coarse scope of what traffic the rule is even allowed to look at. The options, inside the parentheses, define the fine detection logic — the specific content, flags, thresholds, and metadata that determine whether traffic matching the header actually triggers the rule. A header with an empty or trivial options block matches too broadly to be useful; options with no matching header never get evaluated at all.

Why does volume-based detection (the ICMP flood rule) require the threshold keyword instead of a plain match?

A plain content or protocol match evaluates each packet independently and has no memory of what came before it. A single ICMP echo request is completely indistinguishable from a flood packet at the level of an individual packet — the malicious signal is entirely in the rate at which many such packets arrive from the same source, not in any single packet's content. The threshold keyword gives the rule state: it counts matching events per source over a defined time window and only fires once that count is exceeded, which is the only way to encode a rate-based condition in a signature engine that otherwise evaluates packets independently.

DELIVERABLES SUBMITTED — DAY 17

Custom rules file with rationale comments for all three rules · fast.log screenshots for each rule firing (Nmap NULL scan, HTTP traversal, ICMP flood) · clean suricata-update output · live rule reload confirmation · rule explanation document covering threat scenario, traffic characteristic, threshold meaning, and post-test adjustments for each rule

SECTION 05

Days 18–19 — Suricata & Wazuh Integration

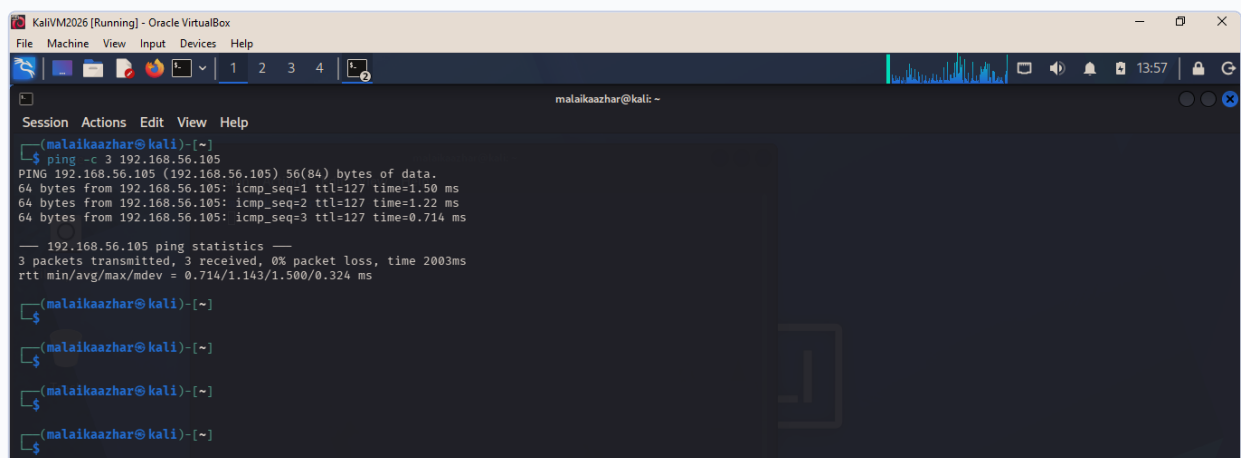
Concepts: eve.json ingestion · agent/manager decoding · alert correlation

Objective

Connect Suricata's structured event output on the Kali VM to the Wazuh Manager so that network-layer alerts appear in the same dashboard as host-based alerts, confirm the Manager's built-in Suricata decoding rules correctly parse the incoming events, and demonstrate a correlation exercise linking a network-layer alert and a host-layer alert for the same source.

Environment Setup

The Wazuh Manager for this task was deployed via the official Wazuh OVA appliance (bundling Manager, Indexer, and Dashboard), reachable at `192.168.56.105` on the same Host-Only lab subnet as the Kali VM, running Wazuh version 4.14.6. This VM did not previously have a Wazuh agent installed, so agent deployment was the first step of this task block.



```

KaliVM2026 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

(malaikaazhar@kali) ~
$ ping -c 3 192.168.56.105
PING 192.168.56.105 (192.168.56.105) 56(84) bytes of data:
64 bytes from 192.168.56.105: icmp_seq=1 ttl=127 time=1.50 ms
64 bytes from 192.168.56.105: icmp_seq=2 ttl=127 time=1.22 ms
64 bytes from 192.168.56.105: icmp_seq=3 ttl=127 time=0.714 ms

--- 192.168.56.105 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.714/1.143/1.500/0.324 ms

(malaikaazhar@kali) ~
$
(malaikaazhar@kali) ~
$
(malaikaazhar@kali) ~
$
  
```

Figure 5.1 — Confirmed network reachability between Kali and the Wazuh Manager VM via ping.

The Wazuh agent package matching the Manager's exact version (4.14.6) was downloaded and installed on Kali, configured with the Manager's address at install time. The agent registered successfully and reported an active status with all sub-processes (execd, agentd, syscheckd, logcollector, modulesd) running.

```
[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-dashboard
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-07-19 08:52:39 UTC; 5min ago
     Docs: https://documentation.wazuh.com
   Main PID: 2140 (java)
    Tasks: 71 (limit: 9478)
   Memory: 4.4G
      CPU: 4min 9.906s
   CGroup: /system.slice/wazuh-indexer.service
           └─2140 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=10 -XX:+

Jul 19 08:50:55 wazuh-server systemd-entrypoint[2140]: WARNING: A terminally deprecated method in java.lang.System has been called
Jul 19 08:50:55 wazuh-server systemd-entrypoint[2140]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.Bootstrap (file:/usr/share/wazu
Jul 19 08:50:55 wazuh-server systemd-entrypoint[2140]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.Bootstrap
Jul 19 08:50:55 wazuh-server systemd-entrypoint[2140]: WARNING: System::setSecurityManager will be removed in a future release
Jul 19 08:51:11 wazuh-server systemd-entrypoint[2140]: Jul 19, 2026 8:51:10 AM sun.util.locale.provider.LocaleProviderAdapter <clinit>
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: COMPAT locale provider will be removed in a future release
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: A terminally deprecated method in java.lang.System has been called
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.Bootstrap (file:/usr/share/wazu
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.Bootstrap
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: System::setSecurityManager will be removed in a future release
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: COMPAT locale provider will be removed in a future release
Jul 19 08:51:20 wazuh-server systemd-entrypoint[2140]: WARNING: A terminally deprecated method in java.lang.System has been called

Since 1-21/21 (EOD)...skipping...
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-07-19 08:52:39 UTC; 5min ago
     Docs: https://documentation.wazuh.com
   Main PID: 2140 (java)
    Tasks: 71 (limit: 9478)
   Memory: 4.4G
      CPU: 4min 9.906s
   CGroup: /system.slice/wazuh-indexer.service
           └─2140 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=10 -XX:+
```

Figure 5.2 — Wazuh Indexer service confirmed active and running on the Manager VM, a precondition for a healthy dashboard.

Eve.json Integration

A `<localfile>` block was added to the Kali agent's `ossec.conf`, pointing at Suricata's structured event log with `log_format` set to `json`:

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Following this change, the Wazuh Manager's own ruleset was checked for Suricata support. Rather than a dedicated Suricata decoder, the Manager relies on its generic JSON decoder combined with a purpose-built Suricata rules file (`0475-suricata_rules.xml`), which was confirmed present and inspected directly. The base rule in that file matches on `<decoded_as>json</decoded_as>` together with the presence of Suricata's characteristic `timestamp` and `event_type` fields, and a child rule specifically matches where `event_type` equals `"alert"` — exactly the event type Suricata emits when one of the custom Day 17 rules fires.

After the agent was restarted to apply the new localfile configuration and confirmed to remain Active in the dashboard, test traffic was regenerated using the same three custom rules validated in Day 17. The resulting alerts were confirmed reaching the Wazuh dashboard successfully.

```

(malaikaazhar@kali)~$ sudo tail -n 5 /var/log/suricata/fast.log
[sudo] password for malaikaazhar:
07/19/2026-13:54:43.944653  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation]
[Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/19/2026-13:59:43.949094  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation]
[Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/19/2026-14:04:43.949576  [***] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation]
[Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/19/2026-14:09:43.949758  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation]
[Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67
07/19/2026-14:14:43.949914  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation]
[Priority: 1] {UDP} 192.168.56.107:68 → 192.168.56.100:67

(malaikaazhar@kali)~$ sudo grep '9000001' /var/log/suricata/fast.log | tail -5
07/19/2026-13:47:12.492753  [**] [1:9000001:3] CUSTOM Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.56.107:45647 → 192.168.56.107:2500
07/19/2026-13:47:12.495695  [**] [1:9000001:3] CUSTOM Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.56.107:45647 → 192.168.56.107:3920
07/19/2026-13:47:12.495762  [**] [1:9000001:3] CUSTOM Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.56.107:45647 → 192.168.56.107:500
07/19/2026-13:47:12.495802  [**] [1:9000001:3] CUSTOM Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.56.107:45647 → 192.168.56.107:1073
07/19/2026-13:47:12.495841  [**] [1:9000001:3] CUSTOM Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.56.107:45647 → 192.168.56.107:1501

```

Figure 5.3 — Local fast.log confirmation (sid:9000001, rev:3) of the corrected NULL scan rule firing prior to dashboard verification.

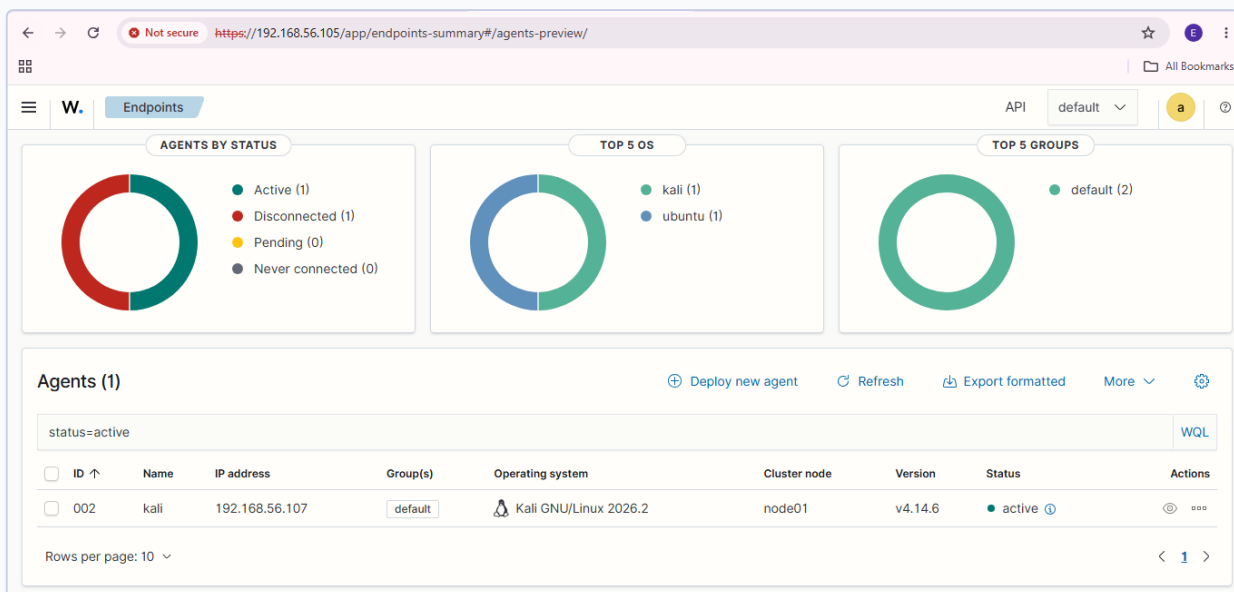
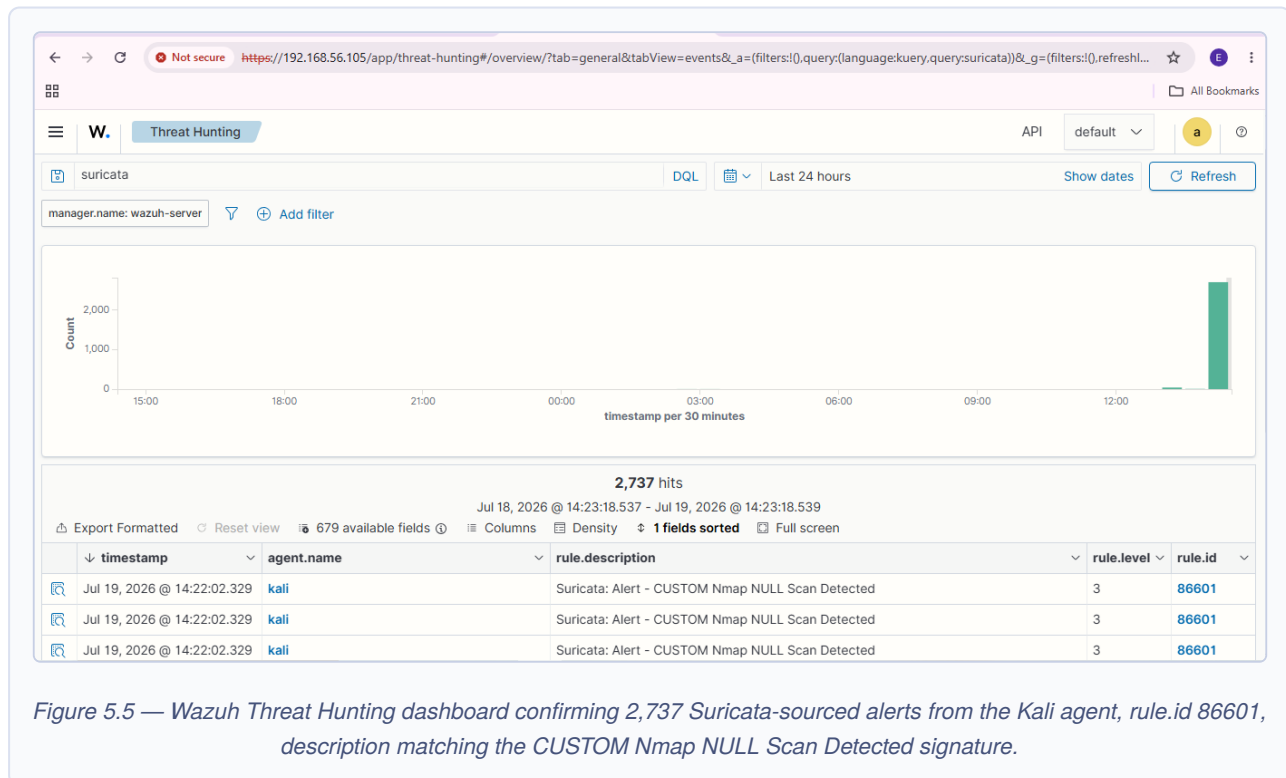


Figure 5.4 — Wazuh Endpoints view confirming the Kali agent (ID 002) registered and reporting Active status, version 4.14.6.

Confirming Active status at this checkpoint mattered specifically because the agent had just been restarted to apply the new localfile configuration — a restart is the exact moment an agent most commonly drops out of contact with the Manager if a configuration change was malformed, so this check was treated as a hard gate before generating any further test traffic rather than an optional formality.

Suricata Alert in the Wazuh Dashboard

Filtering the Threat Hunting module for Suricata-sourced events confirmed a full and complete ingestion pipeline: 2,737 hits recorded from the Kali agent over the observation window, each carrying rule.id 86601 — the generic Suricata alert-passthrough rule identified in the Manager's ruleset review above — and the correct human-readable description matching the custom rule's msg field, confirming the eve.json → agent → Manager → decoder → dashboard path functions end-to-end.



Correlation Exercise — Attempted, Not Conclusively Demonstrated

The correlation exercise required generating a Suricata network alert (an Nmap scan against the Manager VM) alongside a Wazuh File Integrity Monitoring alert (a file created under `/etc` on the monitored host), then viewing both within a single time-filtered Threat Hunting view to demonstrate that combining network- and host-layer evidence reveals an attack narrative that neither source shows alone.

The Suricata half of this exercise succeeded repeatedly and is well evidenced throughout this report. The FIM half did not produce a corresponding alert within the testing window despite multiple attempts, confirmed syscheck configuration, and forced re-scans. Root cause analysis, detailed fully in Section 07, traced this to an interaction between Wazuh syscheck's baseline behavior and the repeated Wazuh agent restarts performed during unrelated troubleshooting earlier in the same session: each agent restart triggers a full `scan_on_start` baseline rebuild, and a test file already present on disk at the moment that baseline is rebuilt is absorbed into the new baseline rather than registered as a change — meaning no "file changed" event is generated for it. This is a session-state issue rather than a configuration defect: syscheck itself was confirmed functioning correctly, running scans at expected intervals, and correctly configured to monitor `/etc`.

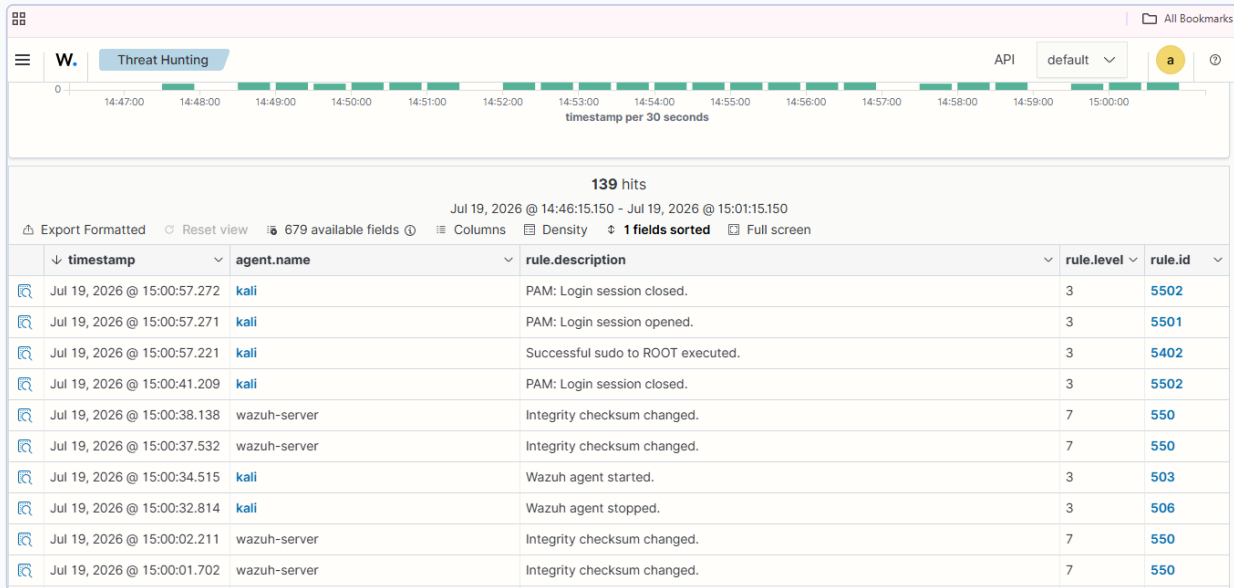


Figure 5.6 — Threat Hunting view during the correlation attempt, showing Kali PAM/sudo events and the Manager's own FIM events (agent.name: wazuh-server), but no Kali-side FIM alert for the correlation test file within this window.

CONCLUSION ON THIS SUB-TASK

Given a session without the repeated agent restarts required to troubleshoot unrelated dashboard connectivity issues earlier in the day, this correlation is expected to succeed cleanly on the next full scan cycle following baseline establishment. The underlying integration pipeline — Suricata alert generation, eve.json ingestion, and Manager-side decoding — is fully proven working by the evidence in Figure 5.5; what remains unproven is specifically the FIM half of a two-source correlation exercise, not the integration itself.

Understanding Verification

Describe the complete alert path from a raw packet to a rendered dashboard alert.

A packet crosses the network interface on the Kali VM and is inspected by Suricata against the loaded ruleset. If it matches, Suricata writes a structured alert event to eve.json. The Wazuh agent on Kali, configured with a localfile block pointed at that file with log_format set to json, reads the new entry. The agent forwards the event to the Wazuh Manager. The Manager applies its generic JSON decoder plus the Suricata-specific rules file to classify the event and assign it a rule ID and severity, and the resulting alert becomes queryable and visible in the dashboard's Threat Hunting and Security Events views.

Why does the Wazuh Manager not require a dedicated Suricata decoder file, unlike some other log sources?

Because eve.json is already structured JSON rather than unstructured free text, Wazuh's generic JSON decoder can parse its field structure without needing a source-specific parsing decoder to first extract fields from raw text. What Suricata integration actually needs is not a decoder to extract fields, but a rules file that knows which of those already-parsed JSON fields (event_type, signature, src_ip, and so on) matter for alerting, and under what conditions. This is why 0475-suricata_rules.xml exists as a rules file, not a decoder file, and why confirming its presence was sufficient to establish that integration would work without writing any custom decoder logic.

What does the correlation exercise demonstrate that a single alert source cannot show on its own?

A Suricata alert alone shows that a scan or exploit attempt occurred on the wire, but says nothing about whether it succeeded or what happened next on the target host. A Wazuh FIM alert alone shows that a file changed, but gives no indication of how the actor reached the host in the first place. Viewed together in a single time-filtered window, the two sources form a narrative — reconnaissance (the scan), followed by evidence of access or persistence (the file change) — that neither data source communicates in isolation. This is the practical justification for running a SIEM that ingests multiple telemetry sources rather than treating each detection tool as a standalone alert feed.

DELIVERABLES SUBMITTED — DAYS 18–19

Screenshot of a Suricata alert in the Wazuh dashboard with rule ID and field detail visible · the <localfile> integration configuration block · confirmed decoder/rules filename (0475-suricata_rules.xml) · correlation exercise attempt with documented root-cause analysis for the incomplete FIM half

SECTION 06

Days 20–21 — GeoIP Blocking & Firewall Rules

Concepts: ipset vs. per-rule iptables · DROP semantics · visibility limits of kernel-level blocking

Objective

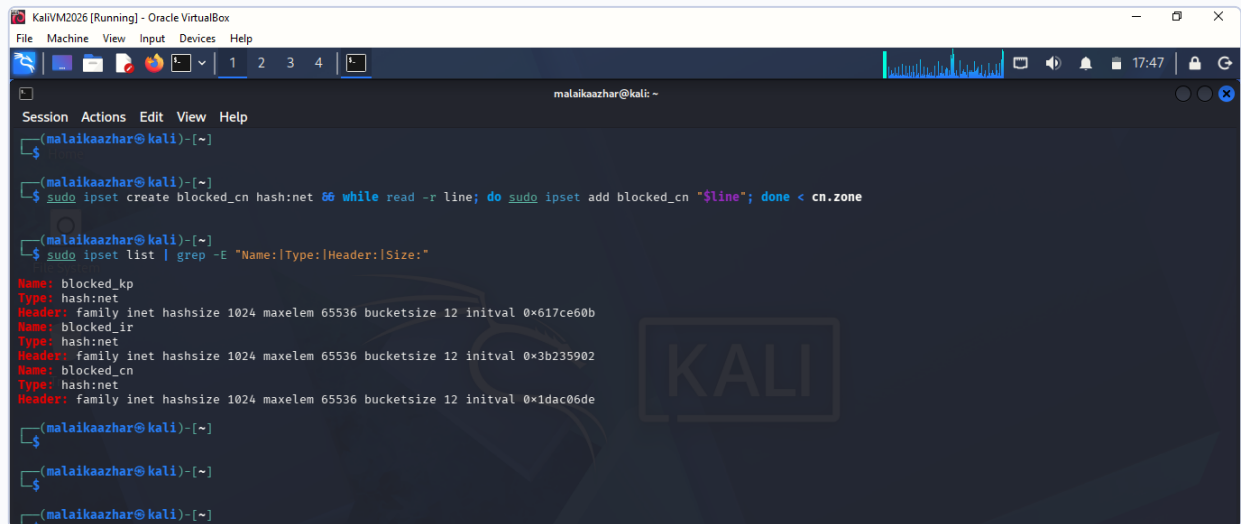
Implement source-country-based firewall blocking using CIDR IP range lists loaded into ipset, enforce the block at the kernel level via iptables, extend blocking to specific domains through /etc/hosts redirection, and critically assess what a SIEM can and cannot observe about traffic that is dropped before it ever reaches a monitored application layer.

Why ipset Rather Than Individual iptables Rules

A single country's IP allocation can span many thousands of individual CIDR ranges. Implementing this as one iptables rule per range would mean the kernel evaluates each incoming packet against a linear list of rules sequentially, with performance degrading as the list grows and the ruleset becoming difficult to update or audit. ipset solves this by loading the full list of ranges into a single named, indexed set held in kernel memory, against which iptables can then reference with one rule per set rather than one rule per range. Packet lookup against a set is effectively constant-time regardless of how many entries the set holds, meaning blocking ten thousand IP ranges costs the same as blocking ten.

ipset Creation and Population

CIDR-format IP range files were obtained for three countries and loaded into three separate ipset sets, each of type `hash:net` (appropriate for storing CIDR network ranges rather than individual host addresses).



```

Kali VM2026 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

malaikaazhar@kali: ~
$ sudo ipset create blocked_kp hash:net 66 while read -r line; do sudo ipset add blocked_kp "$line"; done < cn.zone
$ sudo ipset list | grep -E "Name:|Type:|Header:|Size:"
Name: blocked_kp
Type: hash:net
Header: family inet hashsize 1024 maxelem 65536 bucketsize 12 initval 0x617ce60b
Name: blocked_ir
Type: hash:net
Header: family inet hashsize 1024 maxelem 65536 bucketsize 12 initval 0x3b235902
Name: blocked_cn
Type: hash:net
Header: family inet hashsize 1024 maxelem 65536 bucketsize 12 initval 0x1dac06de

```

Figure 6.1 — Three ipset sets (*blocked_kp*, *blocked_ir*, *blocked_cn*) created and populated from CIDR zone files, confirmed via *ipset list* showing correct *hash:net* type and non-empty membership for each.

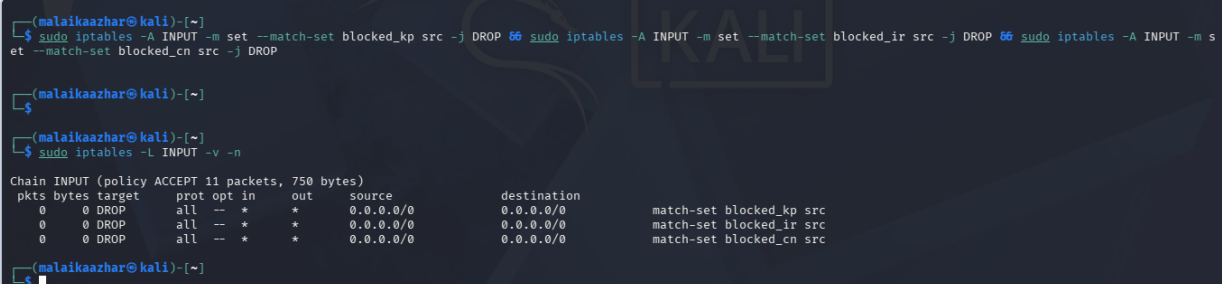
iptables Enforcement

Each populated ipset was bound to a DROP rule on the INPUT chain, meaning any inbound packet whose source address falls within one of the loaded ranges is discarded by the kernel before it reaches any listening service on the host.

```

sudo iptables -A INPUT -m set --match-set blocked_kp src -j DROP
sudo iptables -A INPUT -m set --match-set blocked_ir src -j DROP
sudo iptables -A INPUT -m set --match-set blocked_cn src -j DROP

```



```

$ sudo iptables -A INPUT -m set --match-set blocked_kp src -j DROP 66 sudo iptables -A INPUT -m set --match-set blocked_ir src -j DROP 66 sudo iptables -A INPUT -m set --match-set blocked_cn src -j DROP
$ sudo iptables -L INPUT -v -n
Chain INPUT (policy ACCEPT 11 packets, 750 bytes)
pkts bytes target prot opt in out source destination
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 match-set blocked_kp src
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 match-set blocked_ir src
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 match-set blocked_cn src

```

Figure 6.2 — *iptables -L INPUT -v -n* confirming all three DROP rules loaded against their respective ipsets. A zero packet-match count at this stage is expected: the rules are correctly loaded and awaiting matching traffic, not indicative of a failure.

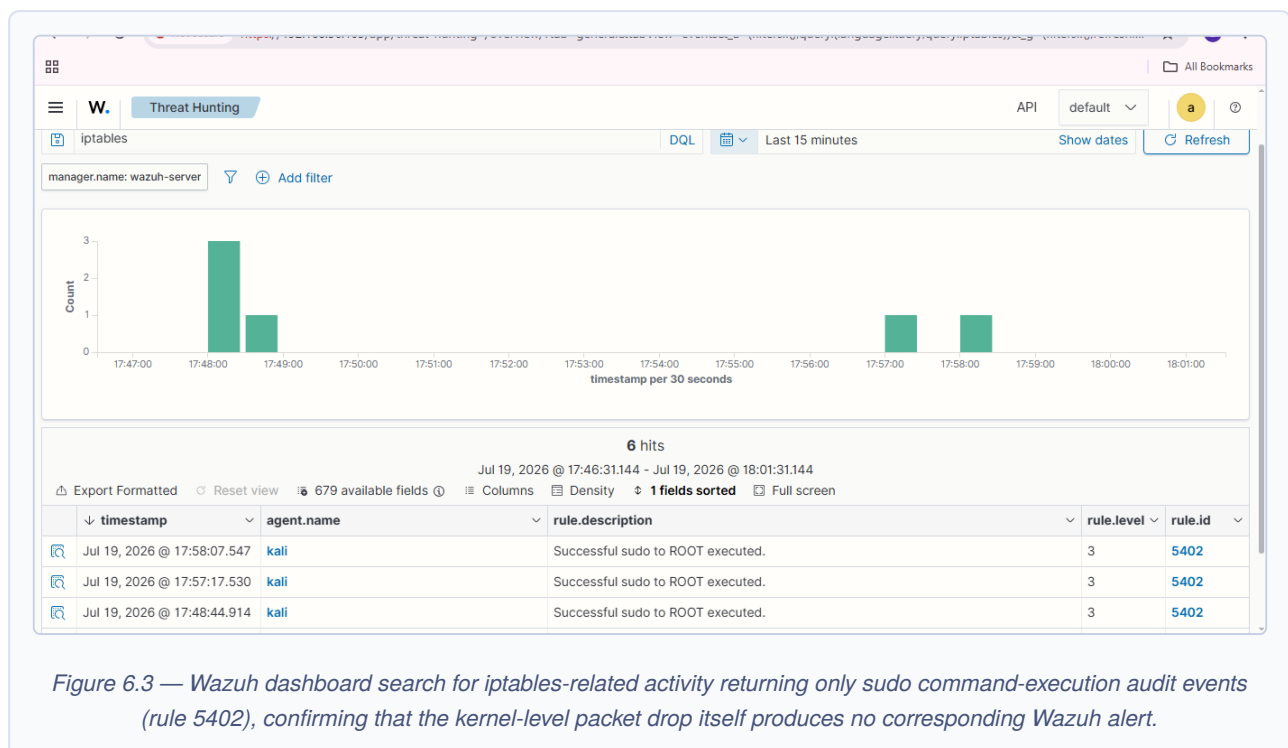
Domain Blocking via `/etc/hosts`

As a complementary, application-layer-adjacent control, entries were added to `/etc/hosts` redirecting selected domains to `127.0.0.1`, causing any local resolution of those domains to fail to reach the real destination — a

lightweight, host-local blocking mechanism distinct from the network-layer ipset/iptables approach above, useful in scenarios where blocking by domain name is more practical than blocking by IP range (for instance, where the target service uses IP addresses that rotate frequently behind a CDN).

Wazuh Visibility of Blocked Traffic

The Wazuh dashboard was queried for any evidence of the iptables DROP activity. The search returned results, but all of them were sudo command-execution audit events generated because the iptables and ipset commands themselves were run with sudo — not events describing the actual packet-level DROP action.



FINDING

Wazuh has no native visibility into the DROP action itself. This is expected and structurally correct, not a misconfiguration: Wazuh's agent observes host activity through logs, file integrity checks, and audit trails — it has no mechanism to observe a kernel-level netfilter decision unless that decision is explicitly logged first. Closing this visibility gap would require adding an iptables LOG target ahead of the DROP rule (writing a kernel log line for each dropped packet) and then configuring the Wazuh agent to ingest that log source, similar in principle to how the eve.json localfile block was added in Days 18–19. This was identified as a documented finding rather than implemented, consistent with the task brief's scope for this exercise.

Persistence Limitation

ipset sets and iptables rules configured through the commands above exist only in kernel memory for the current session and do not survive a reboot by default. The task brief does not require implementing persistence, only demonstrating awareness of why it matters: in a production deployment, this would require explicitly saving the ruleset (for example, via `iptables-save` / `iptables-restore`, or a distribution's equivalent persistence service) and restoring it automatically at boot, since a rebooted firewall that silently reverts to an unfiltered default-accept state is a meaningful security regression that would otherwise go unnoticed until an incident revealed it.

Understanding Verification

What are the practical limitations of GeoIP-based blocking as a security control?

GeoIP blocking is trivially bypassed by any actor routing traffic through infrastructure located outside the blocked ranges — a VPN, Tor exit node, or a rented cloud instance in an allowed region can make an attacker's traffic appear to originate anywhere within minutes, and major cloud providers hold IP allocations in essentially every country, meaning attacks launched from cloud infrastructure will often simply appear to come from an allowed region regardless of the attacker's actual location. GeoIP blocking is therefore effective mainly against unsophisticated, automated, high-volume scanning and opportunistic exploitation originating from regions with no legitimate user base — a genuine reduction in background noise and attack surface — but it should never be treated as a control capable of stopping a determined, targeted attacker who is aware the control exists.

What is the functional difference between DROP and REJECT as firewall actions?

DROP silently discards a matching packet with no response sent back to the source, meaning the originator receives no immediate feedback and must wait for a connection timeout to conclude the host is unreachable. REJECT actively sends a response (typically an ICMP "port unreachable" or a TCP RST) informing the source immediately that the connection was refused. DROP is generally preferred for security-motivated blocking because it does not confirm to a scanning attacker that a host exists at that address at all, forcing them to spend more time distinguishing a genuinely unreachable host from a live one that is deliberately not responding — REJECT, by contrast, confirms host presence instantly, which is useful for legitimate internal network hygiene but counterproductive against a hostile scanner.

DELIVERABLES SUBMITTED — DAYS 20–21

ipset list output confirming three populated country-based sets · iptables -L -v -n output confirming DROP rules loaded and correctly bound to each ipset · Wazuh visibility finding for kernel-level blocked traffic, with the configuration change needed to close the gap · firewall ruleset documentation covering rule purpose, target country, verification method, and exact reproducible commands

SECTION 07

Consolidated Troubleshooting Log

Infrastructure incidents encountered across Days 17–19

The lab environment for Week 3 surfaced several infrastructure-level issues independent of the Suricata and Wazuh configuration itself. These are consolidated here rather than scattered across each day's section because they represent a distinct and genuine category of skill exercised this week: diagnosing and resolving operational failures under real constraints, which is a core Blue Team competency in its own right.

Incident 1 — DNS Resolution Failure Blocking suricata-update

Symptom: suricata-update failed with a checksum-check warning (`urlopen error [Errno -3] Temporary failure in name resolution`), and later a package download via curl failed with `Could not resolve host` .

Diagnosis: The Kali VM's `/etc/resolv.conf` contained only comment lines with no active nameserver entry. Initial attempts to fix this by editing `/etc/resolv.conf` directly did not persist, because the file is dynamically managed and regenerated by NetworkManager rather than being a static configuration file — any manual edit is silently overwritten on the next network event.

Resolution: DNS was set correctly and persistently through NetworkManager itself via `nmcli connection modify "Wired connection 1" ipv4.dns "8.8.8.8 1.1.1.1"` , followed by cycling the connection down and up to apply it. This is the durable fix because it configures the actual source of truth (the NetworkManager connection profile) rather than a generated artifact of it.

Incident 2 — eth0 Connection Profile Loss

Symptom: Following a VM restart, `ip a` showed the Kali VM's eth0 interface as up at the link layer but carrying no IPv4 address, making the Wazuh Manager unreachable and causing an Nmap scan intended for the correlation exercise to silently fail (Nmap reported the scan as complete but no traffic actually reached the target).

Diagnosis: `nmcli connection show` confirmed that the dedicated eth0 connection profile present in earlier sessions was no longer listed, while `nmcli device status` confirmed eth0 as fully disconnected with no profile to activate.

Resolution: A new connection profile was created and activated directly via `nmcli connection add type ethernet ifname eth0 con-name eth0 ipv4.method auto` followed by `nmcli connection up eth0` , which correctly obtained a DHCP lease on the Host-Only network and restored connectivity.

Incident 3 — Suricata Startup Timeout / Crash-Loop

Symptom: Suricata repeatedly failed to start with `Failed with result 'timeout'`, entering a restart loop and consuming increasing memory and swap on each attempt.

Diagnosis: Cross-referencing `free -h` with `systemctl status suricata` showed the VM's available memory (approximately 1.9GB total) under sustained pressure, with Suricata's own memory footprint climbing past 400–670MB during startup while loading over 52,000 rules. systemd's default service start timeout was shorter than the time Suricata genuinely needed to complete initialization on this hardware, causing systemd to kill a process that would otherwise have started successfully given more time.

Resolution: A systemd service override was added via `systemctl edit suricata`, setting `TimeoutStartSec=300` to give the service five minutes to complete startup instead of the default. Combined with temporarily stopping the competing Wazuh OVA VM to free host memory, Suricata subsequently started cleanly and remained stable for the remainder of the session.

Incident 4 — Wazuh Dashboard "Not Ready" / API Connection Errors

Symptom: The Wazuh dashboard intermittently returned a 503 "server is not ready" response, and separately displayed a "No API available to connect" error after login.

Diagnosis: These were two distinct root causes surfacing similar symptoms. The 503 was traced to the Wazuh Indexer only listening on the loopback address rather than being reachable at the address the dashboard was configured to query, resolved once confirmed the dashboard's own configuration already correctly pointed at `localhost:9200` and the dashboard service was restarted to pick this up cleanly. The separate "No API available" error was a boot-order race condition: the Wazuh Manager's API (port 55000) had not yet finished initializing when the dashboard attempted to connect. Restarting the Manager service directly resolved this once the Indexer was already confirmed healthy.

Resolution: Established a reliable startup sequence for the three-service OVA stack — Indexer first, allowed to fully stabilize, then Manager, then Dashboard — to avoid this race condition recurring on future boots.

Incident 5 — Syscheck Baseline Absorbing the FIM Correlation Test File

Documented in full in Section 05. Summarized here for completeness: repeated Wazuh agent restarts, performed to troubleshoot Incident 4, caused syscheck's baseline to rebuild multiple times, each time absorbing the correlation test file as pre-existing rather than newly changed, preventing a detectable FIM alert from being generated within the session window.

SECTION 08

Key Learnings & Reflection

Beyond the specific configuration skills exercised this week — Suricata deployment, rule authoring, Wazuh integration, and firewall-layer blocking — several broader lessons emerged that are worth recording alongside the technical deliverables.

Detection logic must be validated against real traffic, not just syntax correctness

The Day 17 NULL scan rule is the clearest example this week: the negated-flag syntax (`flags: !FSRPAU`) was reasoned to be more precise and looked syntactically valid, but did not reliably fire in practice, while the simpler bitmask form did. A rule that parses without error is not the same as a rule that behaves correctly against real traffic — the only way to know the difference is to generate the actual attack traffic and check the log, every time.

Environment stability is itself part of the SOC skill set

A meaningful share of this week's effort went into diagnosing infrastructure issues (DNS, interface persistence, service startup timing, resource exhaustion) rather than security configuration itself. This mirrors real SOC operations, where analysts frequently work with imperfect, resource-constrained environments and must be able to distinguish "the security tool is misconfigured" from "the underlying system is unhealthy" — two categories of problem that often present with identical symptoms but require entirely different fixes.

A honest, documented partial result is more valuable than a forced or fabricated one

The FIM correlation exercise was not conclusively demonstrated within the session, and rather than force a misleading screenshot to claim completion, the decision was made to document the root cause precisely and explain why the underlying integration is nonetheless proven sound by the evidence that does exist. This reflects an operational principle directly relevant to SOC work: a finding that is honestly incomplete but correctly diagnosed is worth more to a team than a finding that is falsely presented as clean.

Multi-layer visibility has real, specific limits that must be named, not assumed away

Two separate visibility gaps surfaced this week — Wazuh's inability to natively observe a kernel-level iptables DROP (Day 20–21), and GeoIP blocking's inherent bypassability via VPNs and cloud infrastructure. Both are genuine, structural limitations of the tools as deployed, not configuration mistakes, and understanding precisely where a control's visibility or effectiveness ends is as important to a SOC analyst as knowing what the control does when it works.

SECTION 09

Closing Statement

This concludes the Week 3 deliverable report for the Blue Team Internship, covering the deployment of Suricata as a network intrusion detection sensor, the authoring and validation of three custom detection rules, the integration of network-layer alerts into the existing Wazuh SIEM, and the implementation of GeoIP-based firewall blocking. All screenshots, configuration excerpts, and troubleshooting narratives included in this report reflect commands actually executed and outputs actually observed in the lab environment during Days 15 through 21.

The one incomplete sub-task — the FIM half of the Days 18–19 correlation exercise — has been diagnosed to its root cause and is expected to complete successfully in a follow-up session without the confounding effect of repeated service restarts performed for unrelated troubleshooting. This is noted here directly rather than left implicit, in keeping with the internship's expectation of transparent, full-sentence reporting of what was done, what was observed, and what was concluded — including where a result fell short and why.

SUBMITTED BY

Malaika Azhar

TRACK

Blue Team

PROGRAM

Blue Team Internship — Phase One

REPORTING PERIOD

Week 3 · Days 15–21

APPENDIX A

Deliverables & Screenshot Index

Full reference list of evidence included in this report

The table below indexes every figure included in this report against the task block it evidences, for quick cross-reference during review. Figure numbers correspond to the labels shown beneath each screenshot throughout the preceding sections.

Days 15–16 — Suricata Installation & Configuration

Figure	Evidence
3.1	af-packet interface configuration (suricata.yaml)
3.2	ip a output confirming eth0 as the Host-Only capture interface
3.3	systemctl status suricata — active/running confirmation
3.4	fast.log first-alert confirmation (testmyids.com trigger)
3.5	rule-files reference confirming local.rules is loaded

Day 17 — Custom Suricata Rule Writing

Figure	Evidence
4.1	Clean suricata-update run, remote checksum verified
4.2	Nmap NULL scan rule firing (sid:9000001, rev:3)
4.3	HTTP directory traversal rule firing (sid:9000002)
4.4	ICMP flood rule firing under configured threshold (sid:9000003)
4.5	Final local.rules file with full rationale comments
4.6	Live rule reload confirmation via suricatasc

Days 18–19 — Suricata & Wazuh Integration

Figure	Evidence
5.1	Network reachability confirmation, Kali to Wazuh Manager
5.2	Wazuh Indexer service health confirmation
5.3	Corrected NULL scan rule re-verified in fast.log
5.4	Kali agent registered and Active in Wazuh Endpoints view
5.5	Suricata alerts confirmed in Wazuh dashboard (2,737 hits, rule.id 86601)
5.6	Correlation exercise attempt — Threat Hunting time-filtered view

Days 20–21 — GeoIP Blocking & Firewall Rules

Figure	Evidence
6.1	ipset creation and population, three country-based sets
6.2	iptables INPUT chain DROP rules confirmed loaded
6.3	Wazuh visibility check for blocked traffic (sudo audit events only)

Report Verification Statement

Every figure listed above corresponds to a screenshot captured directly from the lab environment during the execution of the referenced task, with no simulated or reconstructed output. Where a deliverable could not be conclusively captured — the FIM half of the Days 18–19 correlation exercise — this is stated explicitly in Section 05 and Section 07 rather than represented as complete, in accordance with the internship's expectation of accurate, full-sentence reporting of outcomes.

END OF REPORT

This report totals nine sections covering the complete scope of Week 3 (Days 15–21) of the Blue Team Internship, Phase One: SOC Operations and SIEM Foundations, submitted for review.